1)
```
console.log(typeof null);
```
2)
```
let a = 1;
let b = 0;
while (a <= 3) {
      a++;
      b += a * 2; 4
      console.log(b);
}
```
3)
```
const employees = [
      { id: 1, name: 'John', department: 'IT', salary: 50000, age: 28, active:
true },
      { id: 2, name: 'Jane', department: 'HR', salary: 45000, age: 32, active:
true },
      { id: 3, name: 'Bob', department: 'IT', salary: 60000, age: 35, active:
false },
      { id: 4, name: 'Alice', department: 'Finance', salary: 55000, age: 29,
active: true },
      { id: 5, name: 'Charlie', department: 'IT', salary: 48000, age: 26,
active: true }
];

let groupdData = {}
employees.forEach(emp => {
      const { active, ...rest } = emp;
      if (!groupdData[emp.department]) {
            emp[department] = [];
      }
      emp[department].push(rest)
});
console.log(groupdData);
```

4)
```
const nums = [1, 2, 3, 4, 5, 6, 7];
nums.forEach((n) => {
      if (n % 2 === 0)
            break;
      console.log(n);
});
```
5)
```
let a = true + true + true * 3; // output I need
```

6)
```
let a = [1, 2, 3, 4, 5];
console.log(a.slice(2, 4));
```

7)
```
const arr = ['A', 'N', 'U'];
arr[10] = 10;
console.log(arr.length);
```
8)
```
isNaN(true)
```

9)
```
const shoes = [
      { name: "Shoe 1", price: 100 },
      { name: "shoe 2", price: 200 },
      { name: "shoe 3", price: 300 }
];
```

10) find the average price of shoes
```
function findAvg(arr) {
```

```
      arr.map((x) => {
            return x.reduce((y) => return x.prrice + y.price)
      })
      return arr[0] / 3;
}

const newShosws = findAvg(shoes)
```

11)

```
function Counter() {

      this.count = 0;

      setInterval(function () {

            this.count++;

            console.log(this.count);

      }, 1000);

}

new Counter();; what will be printed ?
```

12)

```
const employees = [
      { name: "Amit", age: 29, salary: 50000, department: "Engineering" },
      { name: "Neha", age: 32, salary: 47000, department: "Marketing" },
      { name: "Ravi", age: 35, salary: 52000, department: "Sales" },
      { name: "Kavita", age: 28, salary: 43000, department: "Design" },
      { name: "Manish", age: 49, salary: 55000, department: "Finance" },
      { name: "Sneha", age: 30, salary: 46000, department: "HR" },
      { name: "Arun", age: 27, salary: 48000, department: "Support" },
      { name: "Divya", age: 34, salary: 51000, department: "Engineering" }
];
```

i) find the employee with the highest salary
Answer:
  ========

```
      let maxSalary = employees[0];

for (let i = 0; i < employees.length; i++) {
      if (employees[i].salary > maxSalary.salary) {
            maxSalary = employees[i];
      }
}
console.log(maxSalary)
```

ii) find the employee with the lowest age
Answer:
  ========

```
      let minAge = employees[0];

for (let i = 0; i < employees.length; i++) {
      if (employees[i].age < minAge.age) {
            minAge = employees[i];
      }
}
```

```
console.log(minAge);
```

iii) find the average salary of employees
Answer:
  ========

```
      let totalSalary = employees.reduce((sum, emp) => sum + emp.salary, 0);
let averageSalary = totalSalary / employees.length;
console.log(averageSalary);
```

iv) find the total number of employees in each department
Answer:
 =========

```
      let departmentCount = {};
employees.forEach(emp => {
      if (!departmentCount[emp.department]) {
            departmentCount[emp.department] = 0;
      }
      departmentCount[emp.department]++;
});
```

v) find the employee with the longest name
Answer: -
 ==========

```
let longestNameEmployee = employees[0];
for (let i = 0; i < employees.length; i++) {
      if (employees[i].name.length > longestNameEmployee.name.length) {
            longestNameEmployee = employees[i];
      }
}
```

vi) find the average age of employees
Answer:

```
let totalAge = employees.reduce((sum, emp) => sum + emp.age, 0);
let averageAge = totalAge / employees.length;
console.log(averageAge);
```

vi) find the employee with the highest age
Answer:
 =========

```
      let maxAge = employees[0];
for (let i = 0; i < employees.length; i++) {
      if (employees[i].age > maxAge.age) {
            maxAge = employees[i];
      }
}
console.log(maxAge);
```

vii) find the total salary of employees in the "Engineering" department
Answer:
 =========

```
      let engineeringTotalSalary = employees
            .filter(emp => emp.department === "Engineering")
            .reduce((sum, emp) => sum + emp.salary, 0);
console.log(engineeringTotalSalary);
```

viii) find the employee with the highest salary in the "HR" department
Answer:
 =========

```
      let hrEmployees = employees.filter(emp => emp.department === "HR");
let highestHREmployee = hrEmployees[0];
for (let i = 0; i < hrEmployees.length; i++) {
      if (hrEmployees[i].salary > highestHREmployee.salary) {
            highestHREmployee = hrEmployees[i];
      }
```

```
}
console.log(highestHREmployee);
```

ix) find the total number of employees with a salary greater than 50000
Answer:
=========

```
      let highSalaryCount = employees.filter(emp => emp.salary > 50000).length;
console.log(highSalaryCount);
```

x) find the employee with the lowest salary in the "Sales" department
Answer:
=========

```
      let salesEmployees = employees.filter(emp => emp.department === "Sales");
let lowestSalesEmployee = salesEmployees[0];
for (let i = 0; i < salesEmployees.length; i++) {
      if (salesEmployees[i].salary < lowestSalesEmployee.salary) {
            lowestSalesEmployee = salesEmployees[i];
      }
}
```

xi) find the total number of employees in the "Marketing" department
Answer:
=========

```
      let marketingCount = employees.filter(emp => emp.department ===
"Marketing").length;
console.log(marketingCount);
```

13) without using third variable and destrcuring I need to swap the 2 numbers ?
Answer :
=========

```
      let a = 13;
let b = 8
a = a + b;
b = a - b;
a = a - b
console.log(a, b)
```

14) find the longest word in a string;
Answer:
=========

```
var name = "I am from Angular and I am learning JavaScript";
var res = name.split(" ");
var finalResp = res[0];
for (let i = 0; i < res.length; i++) {
      if (res[i].length > finalResp.length) {
            finalResp = res[i];
      }
}
```

15) sum of array elements in a given array
Answer:
=========

```
   var arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
   var sum = 0;
   for (let i = 0; i < arr.length; i++) {
        sum += arr[i];
   }
   console.log(sum);
```

16) I need to find the numbers in an array that end with 7 or 5?
Answer:
==========

```
var arr = [45, 87, 97, 5, 6, 7, 9, 55, 45, 95];
var result = [];
```

```
for (let i = 0; i < arr.length; i++) {
      if (arr[i] % 10 === 7 || arr[i] % 10 === 5) {
            result.push(arr[i]);
      }
}
```

17) I need to find the numbers in an array that are prime numbers?
Answer:
==========

```
var arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
var primeNumbers = [];

function isPrime(num) {
    if (num <= 1) return false;
    for (let i = 2; i <= Math.sqrt(num); i++) {
        if (num % i === 0) return false;
    }
    return true;
}
```

18) I need to find the numbers in an array that are even numbers?
Answer:
==========

```
var arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
var evenNumbers = [];
for (let i = 0; i < arr.length; i++) {
      if (arr[i] % 2 === 0) {
            evenNumbers.push(arr[i]);
      }
}
console.log(evenNumbers);
```

19) I need to find the numbers in an array that are odd numbers?
Answer:
==========

```
var arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
var oddNumbers = [];
for (let i = 0; i < arr.length; i++) {
      if (arr[i] % 2 !== 0) {
            oddNumbers.push(arr[i]);
      }
}
console.log(oddNumbers);
```

20) I need to find the top 4 elements in an array?
Answer:
==========

```
var arr = [9, 4, 1, 7, 3, 9,10,15,9,8];

function findTopFour(arr) {
  let first = -Infinity;
  let second = -Infinity;


  for (let i = 0; i < arr.length; i++) {
    let num = arr[i];

    // Skip duplicates
    if (num === first || num === second ) {
      continue;
    }

    if (num > first) {
```

```
      second = first;
      first = num;
    } else if (num > second) {
      second = num;
    }
  }

  return {
    first: first !== -Infinity ? first : null,
    second: second !== -Infinity ? second : null,

  };
}

console.log(findTopFour(arr));
```

21) I need to get the let values with in the function scope?
Answer:
==========

```
function getLetValues() {
  let a = 10;
  let b = 20;

  return { a, b };
}

const result = getLetValues();
console.log(result); // { a: 10, b: 20 }
```

22) I have name as per the code below?
Answer:
==========

```
  let name = "srinivas";
  name.reverse(); // I need to reverse the name
  console.log(name); // Output: "sivanirs" but here getting error
  // Answer:
  [...name].reverse().join(''); // Correct way to reverse a string in JavaScript
```

23) I need to get the elemenst in the Objects as belwo?
  Answer:
==========

```
var Obj = {name: "srinivas", age: 30, addres: { city: "Hyderabad", state:
"Telangana" }};
  // I need to get the name, age and address in the object by using only
destructuring
  const { name, age, addres: { city, state } } = Obj;
  console.log(name, age, city, state); // Output: srinivas 30 Hyderabad
Telangana
```

24) Closures concept in JavaScript?
Answer:
==========

```
function outer() {
  let count = 0;
  return function inner() {
    return ++count;
  };
}
const counter = outer();
console.log(counter()); // ?
console.log(counter()); // ?
```

ðŸ“˜ Explanation:--
==========
1

2

The inner function remembers the count variable from outer() due to closure.

25) Hoisting in javascript
Answer:
==========
```
console.log(a); // undefined
var a = 10;
```

ðŸ“˜ Explanation:
var is hoisted, but only the declaration, not the value assignment.

26) this in Timeout javascript?
Answer:
==========
```
const person = {
  name: "Alice",
  greet: function () {
    setTimeout(function () {
      console.log("Hello, " + this.name);
    }, 1000);
  }
};
person.greet(); // Output: Hello, undefined
```
ðŸ“˜ Explanation:
i) The `this` keyword inside the setTimeout function does not refer to the `person` object, but to the global object (or undefined in strict mode). To fix this, you can use an arrow function or bind `this`.
ii) Inside setTimeout, this refers to the global object. Use arrow function to retain context.

27) Closures in Loops javascript?
Answer:
==========
```
for (var i = 0; i < 3; i++) {
  setTimeout(function () {
      console.log(i);
  }, 1000);
}
// Output: 3, 3, 3
```
ðŸ“˜ Explanation:
i) The loop completes before the setTimeout callbacks execute, so `i` is 3 in all cases. To fix this, use `let` or an IIFE to create a new scope for each iteration.
ii) var is function-scoped, so all callbacks share same i. Use let for block-scoping.

28) Type Coercion javascript
Answer:
==========
```
console.log(1 + "2"); // "12"
console.log(1 - "2"); // -1
console.log(1 * "2"); // 2
console.log(1 / "2"); // 0.5
console.log(1 + true); // 2
console.log(1 + null); // 1
console.log(1 + undefined); // NaN
console.log([] == false); // true
console.log([] === false); // false
```

ðŸ“˜ Explanation:
== does type coercion: [] is truthy but becomes '', which is falsy. === checks type too.

29) Prototypes javascript?
Answer:-
==========
i)
```
function Person(name) {
      this.name = name;
}
Person.prototype.greet = function() {
      console.log("Hello, " + this.name);
};
const alice = new Person("Alice");
alice.greet(); // Output: Hello, Alice
```

ii)
```
function Animal() {}
Animal.prototype.speak = function () {
  return "Roar";
};
const tiger = new Animal();
console.log(tiger._proto_ === Animal.prototype);   // Output: arduino
```

ðŸ“˜ Explanation:
Every instance's _proto_ points to its constructor's prototype.

30) Async/Await Return javascript?
Answer:
==========
```
async function fetchData() {
  return "data";
}
const data = fetchData();
console.log(data);
```
output:-
=========
Promise {<fulfilled>: "data"}
ðŸ“˜ Explanation:-
==========
Async functions always return a Promise.

31) Async/Await with Error Handling javascript?
Answer:

```
async function fetchData() {
  try {
      // Simulating an API call
      let response = await fetch("https://api.example.com/data");
      if (!response.ok) {
        throw new Error("Network response was not ok");
      }
      let data = await response.json();
      return data;
  } catch (error) {
      console.error("Fetch error:", error);
  }
}
```

32) Promise Chain javascript?
Answer:
```
async function fetchData() {
  return new Promise((resolve, reject) => {
      setTimeout(() => {
        resolve("data");
      }, 1000);
```

```
  });
}
```

33) Async/Await Return javascript?
Answer:

```
async function fetchData() {
  return "data";
}
const data = fetchData();
console.log(data); // Output: Promise {<fulfilled>: "data"}
```

ðŸ“˜ Explanation:
async functions always return a Promise, even if you don't explicitly return one.

34) Promise Chain javascript?
Answer:

```
Promise.resolve(1)
  .then((x) => x + 1)
  .then((x) => { throw new Error("Oops") })
  .catch((err) => console.log(err.message))
  .then(() => console.log("After error"));

// Output:
Oops
After error
```

ðŸ“˜ Explanation:
Error is caught, and chain continues.

35) Array reduce javascript?
Answer:

```
const numbers = [1, 2, 3, 4, 5];
const sum = numbers.reduce((accumulator, currentValue) => {
      return accumulator + currentValue;
}, 0);
console.log(sum); // Output: 15
```

ðŸ“˜ Explanation:
The reduce method iterates through the array, accumulating the sum of its elements.

36) Array reduce javascript?

```
const arr = [1, 2, 3, 4];
const result = arr.reduce((acc, curr) => {
  if (curr % 2 === 0) acc.push(curr * 2);
  return acc;
}, []);
console.log(result); [4, 8]
```

ðŸ“˜ Explanation:
Only even numbers are pushed after doubling.

37) Event Loop javascript?
Answer:

```
===========
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("End");
```

Answer:
Start
End
Promise
Timeout

ðŸ“˜ Explanation:
Microtasks (Promises) run before macrotasks (setTimeout).

38) Destructuring javascript?
Answer:
```
const config = { width: 300 };
const { width, height = 100 } = config;
console.log(width, height); // Output: 300 100
```
ðŸ“˜ Explanation:
Destructuring allows unpacking values from objects or arrays into distinct variables. Default values can be provided for missing properties.

ii) height is not in the object, so default is used.

39) Currying javascript?
Answer:
```
function add(a) {
  return function(b) {
      return a + b;
  };
}
const addFive = add(5);
console.log(addFive(10)); // Output: 15
```
ðŸ“˜ Explanation:
Currying transforms a function with multiple arguments into a series of functions that each take a single argument.

ii)sum(10)(20)(30)(40)(); // I need to get the sum of all the numbers
Answer:
==========
```
let sum = a => b => b ? sum(a + b) : a;
console.log(sum(10)(20)(30)(40)()); // Output: 100
```

40) Shallow Copy javascript?
Answer:
```
const original = { name: "John", meta: { age: 30 } };
const copy = Object.assign({}, original);
copy.meta.age = 40;
console.log(original.meta.age);
```
âœ… Answer:
40
ðŸ“˜ Explanation:
Shallow copy doesn't clone nested objects.

41) IIFE javascript
```
(function () {
  var x = 10;
  console.log(x);
})();
console.log(typeof x);
```
âœ… Answer:
10
undefined
ðŸ“˜ Explanation:
x is scoped inside IIFE.

42) Spread and Rest javascript
```
function test(...args) {
  return args.length;
}
console.log(test(1, 2, 3, 4));
```
âœ… Answer:
4
ðŸ“˜ Explanation:

Rest parameter collects all arguments into an array.

43) Freeze javascript?
```
const obj = { a: 1 };
Object.freeze(obj);
obj.a = 10;
obj.b = 20;
console.log(obj);
```
âœ… Answer:
```
{ a: 1 }
```
ðŸ“˜ Explanation:
Frozen object cannot be modified.

44) bind() javascript

```
function sayHi() {
  return Hi, ${this.name};
}
const user = { name: "Bob" };
const greet = sayHi.bind(user);
console.log(greet());
```
âœ… Answer:
```
Hi, Bob
```
ðŸ“˜ Explanation:
bind permanently sets this context.

45) typeof vs instanceof javascript
```
console.log(typeof null);
console.log(null instanceof Object);
```
âœ… Answer:
```
object
false
```
ðŸ“˜ Explanation:
JS bug: typeof null returns "object", but it's not instance of Object.

46) Classes javascript?
```
class Animal {
  speak() {
    return "Generic sound";
  }
}
class Dog extends Animal {
  speak() {
    return "Bark";
  }
}
const pet = new Dog();
console.log(pet.speak());
```
âœ… Answer:
```
Bark
```
ðŸ“˜ Explanation:
Subclass overrides parent method

47) setTimeout + var javascript
```
for (var i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), 100);
}
```
âœ… Answer:
```
4
4
4
```
ðŸ“˜ Explanation:
var is not block-scoped. All log after loop ends.

48) setTimeout + let javascript

```
for (let i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), 100);
}
```

âœ… Answer:
1
2
3
ðŸ“˜ Explanation:
let creates new scope for each iteration.

49) Object.defineProperty javascript
Answer:-
========

```
const obj = {};
Object.defineProperty(obj, "x", {
  value: 42,
  writable: false
});
obj.x = 100;
console.log(obj.x);
```

âœ… Answer: 42
ðŸ“˜ Explanation:
writable: false makes property immutable.

50) typeof NaN javascript

```
console.log(typeof NaN);
```

âœ… Answer: number
ðŸ“˜ Explanation:
NaN is a special type of number.

51) alsy values javascript

```
const falsyValues = [0, '', null, undefined, NaN, false];
console.log(falsyValues.filter(Boolean));
```

âœ… Answer: [] empty array
ðŸ“˜ Explanation:
Boolean() filters out all falsy values.

52) Optional chaining javascript

```
const user = { profile: null };
console.log(user.profile?.name);
```

âœ… Answer: undefined
ðŸ“˜ Explanation:
Safe access using ?..

53) Number precision javascript

```
console.log(0.1 + 0.2 === 0.3);
```

âœ… Answer: false

ðŸ“˜ Explanation:
Floating point precision issue in JS.

54) parseInt weird case javascript

```
console.log(parseInt("08"));
```

âœ… Answer: 8
ðŸ“˜ Explanation:
Modern JS assumes base 10. Older versions treated "08" as octal.

55) In operator javascript

```
const obj = { a: 1 };
console.log("a" in obj);
console.log("b" in obj);
```
âœ… Answer: true, false
ðŸ“˜ Explanation:
Checks property existence.

56) Function length javascript
```
function a(x, y, z) {}
console.log(a.length);
```
âœ… Answer: 3
ðŸ“˜ Explanation:
Function length returns number of defined parameters.

57) typeof function javascript
```
function test() {}
console.log(typeof test);
```
âœ… Answer: function
ðŸ“˜ Explanation:
Only function returns "function" for typeof.

58) given a string I need Title Case it?
Answer:
```
function titleCase(str) {
      return str.toLowerCase().split(" ").map(word =>
word.charAt(0).toUpperCase() + word.slice(1))
      .join(" ");
}
const input = "hello world";
const output = titleCase(input);
console.log(output); // Output: "Hello World"
```

59) I need to count the string characters in a string?
Answer:-
==========
```
var res = {}
const input = "hello world";
for (let i = 0; i < input.length; i++) {
      const char = input[i];
      if (res[char]) {
            res[char]++;
      } else {
            res[char] = 1;
      }
}
```

60) I need to find the longest word in a string?
Answer:-
==========
```
var name = "I am from Angular and I am learning JavaScript";
var res = name.split(" ");
var finalResp = res[0];
for (let i = 0; i < res.length; i++) {
      if (res[i].length > finalResp.length) {
            finalResp = res[i];
      }
}
console.log(finalResp); // Output: "JavaScript"
```

61) input:
```
   let arrA = [
  { id: 1, name: "test1" },
  { id: 2, name: "test2" },
  { id: 2, name: "test3" },
```

```
  { id: 1, name: "test4" },
  { id: 4, name: "test5" },
  { id: 1, name: "test6" },
  { id: 1, name: "test7" },
  { id: 6, name: "test8" },
]
```

o/p:

```
[
  { id: 1, name: 'Hyderabad' },
  { id: 2, name: 'test2' },
  { id: 2, name: 'test3' },
  { id: 1, name: 'Hyderabad' },
  { id: 4, name: 'test5' },
  { id: 1, name: 'Hyderabad' },
  { id: 1, name: 'Hyderabad' },
  { id: 6, name: 'test8' }
]
```

Answer:
==========

```
arrA = arrA.map(item => {
  if (item.id === 1) {
      return { ...item, name: "Hyderabad" };
  }
  return item;
});
console.log(arrA);
```

62) I need to count the parhantheses in a string?
Answer:
==========

```
input: "(()())()))";
function countParentheses(str) {
  let count = 0;
  for (let char of str) {
      if (char === '(') {
        count++;
      } else if (char === ')') {
        count--;
      }
  }
  return count;
}

console.log(countParentheses(input)); // Output: 1
```

63) input:-
=======

```
test1_test2_test3

I need output as:-
=======
ouput:
i) Test1 Test2 Test3
ii) Test3 Test2 Test1
iii) Test2 Test3 Test1
```

Answer:
==========

```
function formatString(input) {
  return input
      .split('_')
      .map(word => word.charAt(0).toUpperCase() + word.slice(1).toLowerCase())
```

```
      .join(' ');
}
```

64) 1.NaN === NaN // false

2.true + false // 1

3."foo" + + "bar" // "fooNaN"
4.[] + [] // ""

```
var employee;

employee = [];

let promise = new Promise((resolve) => {

console.log("A");

resolve("");

});

console.log("B");

let promise1 = new Promise((resolve) => {

  setTimeout(() => {

      console.log("C");

    resolve("");

  }, 1000);

});


console.log("D");
```

65) Input: 123456789
output: ***********789

```
function maskNumber(input) {
  const str = input.toString();
  const maskedPart = '*'.repeat(str.length - 3);
  const visiblePart = str.slice(-3);
  return maskedPart + visiblePart;
}

const input = 123456789;
const output = maskNumber(input);
console.log(output); // *********789
```

66)

```
function greet(name = "Guest") {
 return Hello, ${name};
}

console.log(greet(undefined));
```

.map() â ” transform every item
.filter() â ” keep only what matches

.find() â ” get the first match
.some() â ” returns true if at least one matches
.every() â ” returns true only if all match

67)
```
const arr = [1, 2, 3];
const [,, ...rest] = arr;
console.log(rest);
```

68)
Input:-
==========
```
const obj1 = {};
const obj2 = { key: "b" };
const obj3 = { key: "c" };
obj1[obj2] = 123;
obj1[obj3] = 234;
console.log(obj1[obj2]); // 234
```

69)
Input:-
==========
```
 const employees = [
  { id: 1, name: 'John', department: 'IT', salary: 50000, age: 28, active:
true },
  { id: 2, name: 'Jane', department: 'HR', salary: 45000, age: 32, active:
true },
  { id: 3, name: 'Bob', department: 'IT', salary: 60000, age: 35, active:
false },
  { id: 4, name: 'Alice', department: 'Finance', salary: 55000, age: 29, active:
true },
  { id: 5, name: 'Charlie', department: 'IT', salary: 48000, age: 26, active:
true }
];
```

i)
```
let groupdData = {}
      employees.forEach(emp => {
         const {active, ...rest} = emp;
         if(!groupdData[emp.department]) {
            emp[department] = [];
         }
         emp[department].push(rest)
      });

      console.log(groupdData)
```

ii)
```
const groupedByDept = {};
for (let emp of employees) {
  if (!groupedByDept[emp.department]) {
    groupedByDept[emp.department] = [];
  }
  groupedByDept[emp.department].push(emp);
}
console.log(groupedByDept);
```

o/p:
```
{
  IT: [
    {
      id: 1,
      name: 'John',
      department: 'IT',
      salary: 50000,
```

```
      age: 28,
      active: true
    },
    {
      id: 3,
      name: 'Bob',
      department: 'IT',
      salary: 60000,
      age: 35,
      active: false
    },
    {
      id: 5,
      name: 'Charlie',
      department: 'IT',
      salary: 48000,
      age: 26,
      active: true
    }
  ],
  HR: [
    {
      id: 2,
      name: 'Jane',
      department: 'HR',
      salary: 45000,
      age: 32,
      active: true
    }
  ],
  Finance: [
    {
      id: 4,
      name: 'Alice',
      department: 'Finance',
      salary: 55000,
      age: 29,
      active: true
    }
  ]
}
{
  IT: [
    {
      id: 1,
      name: 'John',
      department: 'IT',
      salary: 50000,
      age: 28,
      active: true
    },
    {
      id: 3,
      name: 'Bob',
      department: 'IT',
      salary: 60000,
      age: 35,
      active: false
    },
    {
      id: 5,
      name: 'Charlie',
      department: 'IT',
      salary: 48000,
```

```
      age: 26,
      active: true
    }
  ],
  HR: [
    {
      id: 2,
      name: 'Jane',
      department: 'HR',
      salary: 45000,
      age: 32,
      active: true
    }
  ],
  Finance: [
    {
      id: 4,
      name: 'Alice',
      department: 'Finance',
      salary: 55000,
      age: 29,
      active: true
    }
  ]
}
```

70) count the strings?
answer:-
==========

```
var name = "aaabbb";
    let count = {}
    for (const ch of name.toLowerCase()) {
        if (count[ch]) {
            count[ch] = count[ch] + 1
        } else {
            count[ch] = 1
        }
    }
    console.log(count);
```

71)

```
var name = "i am from angular" // I need output I Am From Angular
var res = name.split(' ').map(word => word.charAt(0).toUpperCase() +
word.slice(1)).join(' ');
console.log(res);
```

72)

```
  // closer example code

    function outer() {
        var num = 0;
        return function inner() {
            console.log(++num)
        }
    }
    const count = outer();
    count();
    count();
    count();
    count();
```

73)

```
// destrucre in Objects
    const Obj = {
```

```
        name: "Angular",
        exp: 5,
        company: "genpact",
        location: "bng"
    }

    const { name, exp } = Obj;
    console.log(name, exp);

74) // destrucure in Arrays
      const arr = [1, 2, 3, 4, 5];
      const [first, second, ...rest] = arr;
      console.log(first, second, rest); // Output: 1 2 [3, 4, 5]

75) // I need to get the last 3 elements in an array
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const lastThree = arr.slice(-3);
console.log(lastThree); // Output: [8, 9, 10]

76) // I need to get the first 3 elements in an array
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const firstThree = arr.slice(0, 3);
console.log(firstThree); // Output: [1, 2, 3]

77) const shoes = [
        { name: "Shoe 1", price: 100 },
        { name: "shoe 2", price: 200 },
        { name: "shoe 3", price: 300 }
    ];

          1) How can you extract all shoe names into a separate array?
    const namesList = shoes.map(x => x.name);
    console.log(namesList)

         2) How would you calculate the total price of all shoes?
    const total = shoes.reduce((sum, shoe) => sum + shoe.price, 0);
    console.log(total);


    How can you find a shoe with price 200?
    var findShoew = shoes.find(x => x.price === 200);
    console.log(findShoew)

    How do you filter out shoes with price > 150?
    var findShoew = shoes.filter(x => x.price > 150);
    console.log(findShoew)

    How would you sort the array by price descending?
    const findDesc = shoes.sort((a, b) => b.price - a.price);
    console.log(findDesc);

    How would you add a 10% discount to each shoe without mutating the original
array?
    const duplicateArray = JSON.parse(JSON.stringify(shoes));
    const findPercentage = duplicateArray.map(x => ({
        ...x,
        price: x.price - x.price * 0.1
    }));
    console.log(findPercentage)

    How would you group shoes by price range (e.g., <200 and >=200)?
    const grouped = shoes.reduce((acc, shoe) => {
        const key = shoe.price >= 200 ? 'expensive' : 'cheap';
        (acc[key] = acc[key] || []).push(shoe);
```

```
        return acc;
    }, {});
    console.log(grouped);
```

If this data came from an API, how would you validate that price is a number?

```
    const validate = shoes.every(x => typeof (x.price) === 'number');
    console.log(validate);
```

How would you update the price of "shoe 2" to 250?

```
    updateShoe = shoes.filter(x => x.name === "shoe 2" ? x.price = 250 : x);
    console.log(updateShoe)
```

How do you handle if one shoe doesnâ ™t have a price field?

```
    const safeTotal = shoes.reduce((sum, shoe) => sum + (shoe.price || 0), 0);
    console.log(safeTotal)
```

How do you remove duplicates from the array (based on name)?

```
    const removeduplicates = shoes.filter((shoe, index, self) =>
        index === self.findIndex(s => s.name === shoe.name)
    );
    console.log(removeduplicates, 'removeduplicates');
```

find the averages of shoes prices list

```
    const avg = shoes.reduce((sum, shoe) => sum + shoe.price, 0) / shoes.length;
    console.log(avg, 'avg');
```

How would you convert the array into a single string of shoe names?

```
    const shoeNamesString = shoes.map(shoe => shoe.name).join(', ');
    console.log(shoeNamesString); // Output: "Shoe 1, shoe 2, shoe 3"
```

78)

```
    const employees = [
        { name: "Amit", age: 29, salary: 50000, department: "Engineering" },
        { name: "Neha", age: 32, salary: 47000, department: "Marketing" },
        { name: "Ravi", age: 35, salary: 52000, department: "Sales" },
        { name: "Kavita", age: 28, salary: 43000, department: "Design" },
        { name: "Manish", age: 49, salary: 55000, department: "Finance" },
        { name: "Sneha", age: 30, salary: 46000, department: "HR" },
        { name: "Arun", age: 27, salary: 48000, department: "Support" },
        { name: "Divya", age: 34, salary: 51000, department: "Engineering" }
    ];
```

find the maximum salary for the above employees

```
    let maxSalary = employees[0];

    for (let i = 0; i < employees.length; i++) {
        if (employees[i].salary > maxSalary.salary) {
            maxSalary = employees[i];
        }
    }

    console.log(maxSalary, 'maxSalary')
```

find the minimum salary for the above employees

```
    let minSalary = employees[0];

    for (let i = 0; i < employees.length; i++) {
        if (employees[i].salary < minSalary.salary) {
```

```
            minSalary = employees[i];
        }
    }

    console.log(minSalary, 'minSalary');

    // find the gropued lsit

    let groupdData = {}
    employees.forEach(emp => {
        if (!groupdData[emp.department]) {
            groupdData[emp.department] = [];
        }
        groupdData[emp.department].push(emp)
    });

    console.log(groupdData)

    // How do you extract all employee names?
    employees.map(emp => emp.name);

    How do you find the average salary?
    const avg = employees.reduce((sum, emp) => sum + emp.salary, 0) /
employees.length;

    How do you find employees from the Engineering department?
    employees.filter(emp => emp.department === 'Engineering');

    How do you find the employee with the highest salary?
    employees.reduce((max, emp) => emp.salary > max.salary ? emp : max);


    // Advanced Array Logic

    How would you group employees by department?
    employees.reduce((acc, emp) => {
        (acc[emp.department] = acc[emp.department] || []).push(emp);
        return acc;
    }, {});

    How do you sort employees by age in descending order?
    [...employees].sort((a, b) => b.age - a.age);

    How would you check if all employees have salary > 40,000?
    employees.every(emp => emp.salary > 40000);

    How do you count employees in each department?
    employees.reduce((acc, emp) => {
        acc[emp.department] = (acc[emp.department] || 0) + 1;
        return acc;
    }, {});

    // Scenario-based / Practical Questions

    How would you increase salary by 10% for employees under age 30?
    employees.map(emp => emp.age < 30
        ? { ...emp, salary: emp.salary * 1.10 }
        : emp
    );

    How would you paginate this list (2 per page)?
    function paginate(arr, page, size) {
        return arr.slice((page - 1) * size, page * size);
    }
```

```
    How would you get the total salary by department?
    employees.reduce((acc, emp) => {
        acc[emp.department] = (acc[emp.department] || 0) + emp.salary;
        return acc;
    }, {});
```

JavaScript + Architecture Concepts
==========

79) How would you memoize the total salary calculation?

80) How do you ensure immutability while updating salary values?

81) How do you handle large datasets with infinite scroll?

82) How would you validate this data in a form (e.g., in Angular)?

83) How would you connect this employee array to a REST API table grid (Angular Material / AG-Grid)?

84)
```
const user = {
    val: 0,
    increment: () =>  {
        // setTimeout(() => {
            this.val++;
            console.log(this.val);
        // }, 1000);
    }
}

user.increment();
user.increment();

const obj = {
    value: 20,
    getValue:  () => {
        setInterval(() => {
            return ++this.value;
        }, 1000);
    }
};
console.log(obj.getValue())
```

85)
```
var arr = [2, 3, 10] // I need missing the numbers in the array
var maxCountNumber = arr[arr.length - 1];
var missingNumbers = [];

for (let i = 1; i < maxCountNumber; i++) {
    if (!arr.includes(i)) {
        missingNumbers.push(i);
    }
}

console.log(missingNumbers)
```

86)
```
var arr = [1, 2, 3, 1];
var duplicate = [];

for (let i = 0; i < arr.length; i++) {
    if (!duplicate.includes(arr[i]) && arr.indexOf(arr[i]) !== i) {
        duplicate.push(arr[i]); //only duplicate numbers
    }
```

```
}
console.log(duplicate, 'uniqueNums');
```

87)

```
function getCarPrice(model) {
  model = model.toLowerCase();
  // if (model === 'swift') {
  //     return 600000;
  // } else if (model === 'creta') {
  //     return 1000000;
  // } else if (model === 'fortuner') {
  //     return 4000000;
  // } else if (model === 'xuv700') {
  //     return 2200000;
  // } else if (model === 'tiago') {
  //     return 550000;
  // } else {
  //     return 'Model not found';
  // }


  // belwo the process very less code

   const user = {
      'swift': 600000,
      'creta': 1000000,
      'fortuner': 4000000,
      'xuv700': 2200000,
      'tiago': 550000,
  }
  console.log(user[model])

}

getCarPrice('fortuner');
```

88)

```
const arr = [5, 12, 8, 20, 19];

let highest = -Infinity;
let secondHighest = -Infinity;

// for (let i = 0; i < arr.length; i++) {
//     let num = arr[i];

//     if (num > highest) {
//         secondHighest = highest;
//         highest = num;
//     } else if (num > secondHighest && num < highest) {
//         secondHighest = num;
//     }
// }

for (let i = 0; i < arr.length; i++) {
    let num = arr[i];

    if (num > highest) {
        secondHighest = highest
        highest = num;
    } else if (num > secondHighest && num < highest) {
        secondHighest = num;
    }
}
```

```
    console.log("2nd highest number:", secondHighest);


      89)

      const users = {
        frontendmaster: {},
        rahul: {}
    }

    let name = 'constructor';

    if (users[name]) {
        console.log('BOSS');
    } else {
        console.log("else")
        // some other logic
    }

    // what is the output why

      90) var arr = [1, 3, 4, 8, 7, 9, 3, 5, 1]
    var k = 2;  // I need find the diffrence between 2 numbers equal to 2 how
can i
    var res = [];

    for (let i = 0; i < arr.length; i++) {
        for (let j = 1; j < arr.length; j++) {
            if (arr[i] - arr[j] === k) {
                res.push([arr[i], arr[j]]);
            }
        }
    }
    console.log(res)

 91)  This is recurisive function example code
    var res = [];
    function checkArray(arr) {
      for(let i = 0; i<=arr.length-1;i++) {
        if (Array.isArray(arr[i])) {
            checkArray(arr[i]);
        } else {
            res.push(arr[i]);
        }
      }
    }
    checkArray([1,2,[3,4,[5,6]], 7,[8,9]]);
    console.log(res);

92)
// Given an array of employee objects, remove duplicates based on the 'id'
property
const employees = [
    { id: 1, name: 'John', department: 'IT', salary: 50000, age: 28, active:
true },
    { id: 1, name: 'John', department: 'IT', salary: 50000, age: 28, active:
true },
    { id: 2, name: 'Jane', department: 'HR', salary: 45000, age: 32, active:
true },
    { id: 2, name: 'Jane', department: 'HR', salary: 45000, age: 32, active:
true },
    { id: 3, name: 'Bob', department: 'IT', salary: 60000, age: 35, active:
false },
```

```
    { id: 3, name: 'Bob', department: 'IT', salary: 60000, age: 35, active:
false },
    { id: 4, name: 'Alice', department: 'Finance', salary: 55000, age: 29,
active: true },
    { id: 5, name: 'Charlie', department: 'IT', salary: 48000, age: 26, active:
true }
  ];

  // Remove duplicates based on 'id' property
 var res = [];
 employees.forEach((item) => {
     if(!res.some(e => e.id === item.id)) {
       res.push(item);
     }
 });
 console.log(res);
```

93) find the longest word in a string

```
var name = "I am from Angular and I am learning JavaScript";
var res = name.split(" ");
var finalResp = res[0];
for (let i = 0; i < res.length; i++) {
   if(res[i].length > finalResp.length) {
      finalResp = res[i];
   }
}
console.log(finalResp); // Output: "JavaScript"
```

94)

What's the output of the code below - and why? â¬‡

```
const arr = [1, , 3];

const result = arr.map((x) => x * 2);

console.log(result);
```

95)

```
const obj1 = { a: 1, b: 2 };
const obj2 = { b: 3, c: 4 };
```

how can I merge 2 objects

96) oops concepts in JavaScript? with class and function oops concepts